

CppUTest

CppUTest is a lightweight C/C++ unit testing framework designed to test embedded system software on host machines before deploying to physical hardware. Key features include simple test grouping, built-in memory leak detection, and support for mock objects.

Jenkins and Github Action are major tools for CI/CD. These tasks proceed through stages that include initiation, writing unit tests, and verifying that the completed features function correctly.

The most widely used unit test frameworks are below.

Utest	C/C++ header based lightweight test framework
CppUTest	C/C++ based and working on Linux.. Support memory leak check and ,Mock
Google test	Relies on the C++ STL, a de facto standard framework.

I mentioned Mock earlier; their functions can be categorized as follows.

Stub	basic interface implementation that returns predefined values (e.g. always return 0)
Mock	basic interface implementation that performs predefined behavior depending on the input values (e.g. simple if/else statements that give different return values for different inputs)
Fake	limited working implementation that pretends to be a dependency but might not rely on actual underlying hardware or frameworks (e.g. I2C driver that pretends to be a particular temperature sensor).

Hardware-in-the-Loop (HIL) is a frequently mentioned term that refers to the inclusion of hardware as part of the test and verification phase.

A unit test is a process used to verify that a specific implemented feature functions correctly according to requirements. It is an additional step that might seem unnecessary if the feature has already been implemented well. It serves as a minimal verification process for individual errors.

Just because your unit tests passed successfully, do not assume that your code is free of bugs.

It is important to note that unit testing is a stage for verifying required functionality; it cannot guarantee the error free code in a real-world production environment.

In the above listed items, I will explain CppUTest.

You can reference below URLs

https://cpputest.github.io/manual.html https://cpputest.github.io/mocking_manual.html
--

The problem is, even I reviewed the web manuals, it was too difficult to fully understand and implement it on Unit Test. So, I review some other reference and prepare the CppUTest example on below location.

https://github.com/vspauto/utest

Install

This guide assumes that the host OS is Ubuntu. If you are using Windows, consider using Docker.

The easiest way to install CppUTest on Ubuntu is to use the pre-packaged version.

```
$ apt-get install cpputest
```

However, this application note explains how to download the source code and set up the environment. I encourage you to examine not only "cpputest/build/MakefileWorker.mk"—which will be discussed later—but also the other sample code.

First, decide where to install CppUTest on your local host.

I will install CppUTest in a subdirectory named `cpputest` at the location `~/github/utest`.

```
$ cd ~/github/utest
```

```
$ git clone https://github.com/cpputest/cpputest.git
```

```
$ cd cpputest
```

```
$ sudo apt install -y build-essential autoconf automake libtool
```

```
$ autoreconf -i
```

```
$ ./configure
```

```
$ make
```

You can use `make install` if you want to install CppUTest system-wide

Quick Introduction

At first, make “cpputest” directory on your project.

```
~/github/utest/  
+-- cpputest/  
+-- src/  
+-- tests/  
+-- Makefile
```

Your source codes are located on “src” directory, and Unit Test codes will be located on “tests” directory, because, I want to separate the source code and test code.

On the “tests” directory, create component directories that you want to verify. In my case, I generated “average”, “ring_buffer”, and “sensor” components on <https://github.com/vspauto/utest>.

On each component, it needs main.cpp and files that related with your test items.

main.cpp is simple. It just need “CppUTest/CommandLineTestRunner.h” header and “CommandLineTestRunner::RunAllTests()” command.

```
#include "CppUTest/CommandLineTestRunner.h"  
int main(int argc, char* argv[])  
{  
    return CommandLineTestRunner::RunAllTests(argc, argv);  
}
```

The test item file can be any name, for example test.cpp. To prepare, add `#include "CppUTest/TestHarness.h"` and use `TEST_GROUP(group_name)`, end with semicolon(;), and use `TEST(group name, test name)` to create the test items.

```
#include "CppUTest/TestHarness.h"  
  
TEST_GROUP(FirstTestGroup)  
{  
    void setup() {}  
}
```

```

    void teardown() {}
};

TEST(FirstTestGroup, FirstTest)
{
    FAIL("Fail me!");
}

```

If you want to add the second test item, use TEST(group name, test name).

```

TEST(FirstTestGroup, SecondTest)
{
    STRCMP_EQUAL("hello", "world");
    LONGS_EQUAL(1, 2);
    CHECK(false);
}

```

On the TEST(), user can add some assertions, for example CHECK(), CHECK_EQUAL().

You can add several TEST_GROUP() on on component file. For example,

```

#include "CppUTest/TestHarness.h"

TEST_GROUP(FirstTestGroup)
{
};
TEST(FirstTestGroup, test_11)
{
}

TEST_GROUP(SecondTestGroup)
{
};
TEST(SecondTestGroup, test_21)
{
}
TEST(SecondTestGroup, test_22)
{
}

```

MakefileWorker.mk

You can view this file at the following location without installing it.

<https://github.com/cpptest/cpptest/blob/master/build/MakefileWorker.mk>

You can find out, the following variable values are required.

INCLUDE_DIRS	Directories used to search for include files. This generates a -I for each directory
SRC_DIR	Directories containing source files to build into the library
TEST_SRC_DIRS	Directories containing unit test code build into the unit test runner
COMPONENT_NAME	the name of the thing being built , for example, "ring_buffer"
CPPUTEST_HOME	where CppUTest home dir found

You can obtain PROJECT_DIR using the following script,

```
# Dynamically generate absolute paths
```

```
# From https://matheusmbar.com/bugfree-robot/2019/04/05/makefile-first-test.html
MAKEFILE_DIR = $(dir $(realpath $(firstword $(MAKEFILE_LIST))))
PROJECT_DIR = $(realpath $(MAKEFILE_DIR)../..)
SRC_DIR = $(PROJECT_DIR)/src
TEST_DIR = $(PROJECT_DIR)/tests
```

Define the necessary variables as follows. Note that `SRC\_DIR` is reused, and `MakefileWorker.mk` is included at the end.

```
# Name of component under test
COMPONENT_NAME = average

## Location of CppUTest
CPPUTEST_HOME ?= $(PROJECT_DIR)/cpputest

# Include and source files for component under test
INCLUDE_DIRS = $(SRC_DIR)
SRC_DIR = $(SRC_DIR)

# Includes CppUTest header files
INCLUDE_DIRS += $(CPPUTEST_HOME)/include

# Directories containing the unit tests
TEST_SRC_DIRS = $(TEST_DIR)/$(COMPONENT_NAME)

# Run MakefileWorker.mk with the variables set in this Makefile
include $(CPPUTEST_HOME)/build/MakefileWorker.mk
```

Compiler options

```
# path for the CppUTest pre-include header
CPPFLAGS += -I$(CPPUTEST_HOME)/include

# for the memory leak detection
CXXFLAGS += -include $(CPPUTEST_HOME)/include/CppUTest/MemoryLeakDetectorNewMacros.h
CFLAGS += -include $(CPPUTEST_HOME)/include/CppUTest/MemoryLeakDetectorMallocMacros.h

# Linker option
LD_LIBRARIES = -L$(CPPUTEST_HOME)/lib -lCppUTest -lCppUTestExt
```

Setup and Teardown

Every test group can have a setup and a teardown method. The setup method is called *before* each test and the teardown method is called *after* each test.

Memory Leak Detection

CppUTest has memory leak detection support on a per-test level.

```
Pre-setup -> Record the amount of memory used
Do setup
Run test
Do teardown
Post-teardown -> Check whether the amount of memory is the same
```

C Interface

CppUTest is based the cpp code. For the pure C code, at first add extern "C" to link the header file.

```
//On your cpp file
```

```
extern "C" {  
    #include "Your_C_Header_file.h"  
}
```

CppUMock

CppUMock is an extension module built into the CppUTest framework that handles dependency mocking during unit testing. In embedded C/C++ development, it plays a pivotal role in establishing independent testing environments by replacing components—such as hardware control functions, I2C/SPI communication, network/socket APIs, and external C library drivers—with mock functions, thereby eliminating the need for actual hardware or external integrations.

Core Concepts: Expectation and Actual Call

The basic operating principle of CppUMock is very intuitive.

Test Code	You register an "Expectation," specifying which function should be called, with what arguments, how many times, and what it should return.
Mock Function	Called in place of the actual implementation, it records the "Actual Call," noting the arguments with which the function was invoked.
Verification Phase	At the end of the test, it automatically verifies whether the expected calls and the actual calls match perfectly.

Major routines

```
#include "CppUTest/TestHarness.h"  
#include "CppUTestExt/MockSupport.h"  
  
TEST_GROUP(MockDocumentation)  
{  
    void teardown()  
    {  
        mock().clear();  
    }  
};  
  
TEST(MockDocumentation, SimpleScenario)  
{  
    mock().expectOneCall("productionCode");  
    mock().actualCall("productionCode");  
    mock().checkExpectations();  
}
```

Parameters

Recording parameters on a function

```
mock().expectOneCall("function").onObject(object).withParameter("p1",  
2).withParameter("p2", "hah");
```

And the actual call is like:

```
mock().actualCall("function").onObject(this).withParameter("p1",  
p1).withParameter("p2", p2);
```

Output Parameters

in the expected call

```
int outputValue = 4;  
mock().expectOneCall("Foo").withOutputParameterReturning("bar", &outputValue,  
sizeof(outputValue));
```

the actual call

```
void Foo(int *bar)  
{  
    mock().actualCall("Foo").withOutputParameter("bar", bar);  
}
```

Return values

in the expected call

```
mock().expectOneCall("function").andReturnValue(10);  
Return,
```

```
int function () {  
    return mock().actualCall("function").returnIntValue();  
}
```

```
int function () {  
    mock().actualCall("function");  
    return mock().intReturnValue();  
}
```

Pure C Interface

```
#include "CppUTestExt/MockSupport_c.h"  
  
mock_c()->expectOneCall("foo")  
    ->withIntParameters("integer", 10)  
    ->andReturnDoubleValue(1.11);  
return mock_c()->actualCall("foo")  
    ->withIntParameters("integer", 10)  
    ->returnValue().value.doubleValue;
```

Example: Average

Let's assume, you have below function.

project/src/ average/average.h

```
#ifndef AVERAGE_H
#define AVERAGE_H

float average(float arr[], unsigned int size);

#endif
```

project/src/ average /average.c

```
#include "average.h"

float average(float arr[], unsigned int size)
{
    if (size == 0) {
        return 0.0;
    }

    float total = 0;
    for (unsigned int i = 0; i < size; i++)
    {
        total += arr[i];
    }

    return total / (float)size;
}
```

project/src/main.c

```
#include "average/average.h"

int main(int argc, char *argv[])
{
    ...
    float array[] = {-1.0, 0.0, 1.0, 2.0, 3.0};
    float avg = average(array, 5);
    printf("Average: %f\r\n", avg);
    ...
    return 0;
}
```

Let's verify that this function works correctly. Prepare main.cpp and below test.cpp file.

project/tests/average/test.cpp

```
#include "CppUTest/TestHarness.h"

extern "C" {
    #include "average.h"
}

// Create a test group
```

```

TEST_GROUP(average_test_group)
{
};

// Test the average function
TEST(average_test_group, simple_test)
{
    float array[] = {-1.0, 0.0, 1.0, 2.0, 3.0};
    float avg = average(array, 5);
    CHECK_EQUAL(avg, 1.0);
}

// Test null array
TEST(average_test_group, null_test)
{
    float array[1];
    float avg = average(array, 0);
    CHECK_EQUAL(avg, 0.0);
}

```

In `simple\_test`, Provides five input values, and verifies the expected result is returned.

In `null\_test`, a size argument of 0 is provided, and the code checks whether the result is 0.0.

Since the `average()` function is implemented in a `.c` file, `extern "C"` is used to link the header.

Note that because the function is defined in the `average` subdirectory, the `Makefile` must be modified as follows.

```

# Name of component under test
COMPONENT_NAME = average

# Dynamically generate absolute paths
# From https://matheusmbar.com/bugfree-robot/2019/04/05/makefile-first-test.html
MAKEFILE_DIR = $(dir $(realpath $(firstword $(MAKEFILE_LIST))))
PROJECT_DIR = $(realpath $(MAKEFILE_DIR)../..)
SRC_DIR = $(PROJECT_DIR)/src
TEST_DIR = $(PROJECT_DIR)/tests

# Location of CppUTest (if not already set)
CPPUTEST_HOME ?= $(PROJECT_DIR)/cputest

# Include and source files for component under test
INCLUDE_DIRS = $(SRC_DIR)/$(COMPONENT_NAME)
SRC_DIRS = $(SRC_DIR)/$(COMPONENT_NAME)

# Includes CppUTest header files
INCLUDE_DIRS += $(CPPUTEST_HOME)/include

# Directories containing the unit tests
TEST_SRC_DIRS = $(TEST_DIR)/$(COMPONENT_NAME)

# Run MakefileWorker.mk with the variables set in this Makefile
include $(CPPUTEST_HOME)/build/MakefileWorker.mk

```


Example: Ring-Buffer

project/src/ring_buffer.h

```
#ifndef __RING_BUFFER_H__
#define __RING_BUFFER_H__

#include <stdbool.h>
#include <stdint.h>

struct RingBuffer_s {
    int count;
    int head;
    int tail;
    int *buffer;
    int size;
};

typedef struct RingBuffer_s ring_buffer_t;

bool ring_buffer_init(ring_buffer_t *rb, int *buffer, int size);
bool ring_buffer_push(ring_buffer_t *rb, int data);
bool ring_buffer_pop(ring_buffer_t *rb, int *data);

#endif
```

project/src/ring_buffer.c

```
#include <stdbool.h>
#include "ring_buffer.h"

bool ring_buffer_init(ring_buffer_t *rb, int *buffer, int size)
{
    rb->count = 0;
    rb->head = 0;
    rb->tail = 0;
    rb->size = size;
    rb->buffer = buffer;
    return true;
}

bool ring_buffer_is_full(ring_buffer_t *rb)
{
    if(rb->count == rb->size)
        return true;
    return false;
}

bool ring_buffer_is_empty(ring_buffer_t *rb)
{
    if(rb->count == 0)
        return true;
    return false;
}

bool ring_buffer_push(ring_buffer_t *rb, int data)
{
    if(ring_buffer_is_full(rb))
        return false;
    rb->buffer[rb->head] = data;
    rb->head = (rb->head + 1) % rb->size;
    rb->count++;
    return true;
}
```

```

    if (ring_buffer_is_full(rb)) {
        return false; // Buffer is full
    }

    rb->buffer[rb->head] = data;
    rb->head = (rb->head + 1) % rb->size;
    rb->count++;

    return true;
}

bool ring_buffer_pop(ring_buffer_t *rb, int *data)
{
    if (ring_buffer_is_empty(rb)) {
        return false; // Buffer is empty
    }

    *data = rb->buffer[rb->tail];
    rb->tail = (rb->tail + 1) % rb->size;
    rb->count--;

    return true;
}

```

project/src/main.c

```

#include <stdio.h>
#include "ring_buffer.h"

int main(int argc, char *argv[])
{
    ring_buffer_t rb;
    #define RB_SIZE 8
    int buffer[RB_SIZE] = {0};

    printf("\nInit RingBuffer size:%d\n", RB_SIZE);
    ring_buffer_init(&rb, buffer, RB_SIZE);
    printf("--- Pushing items ---\n");
    for (int i = 1; i <= 5; i++) {
        if (ring_buffer_push(&rb, (uint8_t)(i * 10))) {
            printf("Pushed: %d\n", i * 10);
        } else {
            printf("Failed to push %d (Buffer Full)\n", i * 10);
        }
    }

    printf("Current buffer size: %d\n", rb.count);

    printf("--- Popping items ---\n");
    int val;
    while (ring_buffer_pop(&rb, &val)) {
        printf("Popped: %u\n", val);
    }

    return 0;
}

```

```
}
```

Next, let's write test components for the `ring_buffer`.

To use `ring_buffer`, add `ring_buffer_init()` on `setup()` of `TEST_GRPUP()`.

Test items.

1	Check (<code>rb.count == 0</code>) after it executes <code>ring_buffer_init()</code> .
2	Push the value 42 and verify that <code>rb.count</code> has increased. Execute Pop, and then verify that <code>rb.count</code> is decreased, and confirm the returned value 42.
3	For a maximum size of 3, Verify that the fourth `push` operation fails as expected. Perform four `pop` operations and verify that the final `pop` fails as expected.

```
#include "CppUTest/TestHarness.h"
```

```
extern "C" {  
    #include "ring_buffer.h"  
}
```

```
#define BUFFER_SIZE 3
```

```
// Create a test group  
TEST_GROUP(ring_buffer_test_group)  
{  
    ring_buffer_t rb;  
    int raw_buffer[BUFFER_SIZE];  
  
    void setup()  
    {  
        // Initialize before each test  
        ring_buffer_init(&rb, raw_buffer, BUFFER_SIZE);  
    }  
  
    void teardown()  
    {  
        // Deinitialize after each test  
    }  
};
```

```
/* check (rb.count == 0) after it executes ring_buffer_init(). */  
TEST(ring_buffer_test_group, test_InitEmpty)  
{  
    CHECK_EQUAL(0, rb.count);  
    POINTERS_EQUAL(raw_buffer, rb.buffer);  
}
```

```
/*  
Push the value 42 and verify that rb.count has increased.  
Execute Pop, and then verify that rb.count is decreased,  
and confirm the returned value 42.  
*/  
TEST(ring_buffer_test_group, test_PushPop)  
{  
    int out_val = 0;  
  
    CHECK_TRUE(ring_buffer_push(&rb, 42));  
    CHECK_EQUAL(1, rb.count);
```

```

    CHECK_TRUE(ring_buffer_pop(&rb, &out_val));
    CHECK_EQUAL(42, out_val);
    CHECK_EQUAL(0, rb.count);
}

/*
For a maximum size of 3,
Verify that the fourth `push` operation fails as expected.
Perform four `pop` operations and verify that the final `pop` fails as expected.
*/
TEST(ring_buffer_test_group, test_OverflowAndUnderflow)
{
    int out_val = 0;

    //push 3
    CHECK_TRUE(ring_buffer_push(&rb, 10));
    CHECK_TRUE(ring_buffer_push(&rb, 20));
    CHECK_TRUE(ring_buffer_push(&rb, 30));
    // check overflow
    CHECK_FALSE(ring_buffer_push(&rb, 40));

    //pop all 3
    CHECK_TRUE(ring_buffer_pop(&rb, &out_val));
    CHECK_TRUE(ring_buffer_pop(&rb, &out_val));
    CHECK_TRUE(ring_buffer_pop(&rb, &out_val));
    // check underflow
    CHECK_FALSE(ring_buffer_pop(&rb, &out_val));
}

```

Example: I2C Sensor with Mock

Normally, I2C needs Hardware Client. We will use CppMock to emulate I2C sensor that read I2C.

Let's assume, we have a temperature sensor on I2C ID 0x48, and register value on 0x01 represent the temperature. On the below sensor.c file, it returns the temperature value or returns -999 if I2C is failed.

```

#include <stdint.h>
#include "sensor.h"

#define SENSOR_I2C_ADDR 0x48
#define TEMP_REG_ADDR 0x01

int read_temperature_sensor(void)
{
    uint8_t raw_temp = 0;
    bool result;

    result = I2C_read(SENSOR_I2C_ADDR, TEMP_REG_ADDR, &raw_temp, 1);
    if(result != true) {
        return -999; /* ERROR */
    }
    int celsius = (int)raw_temp;
}

```

```

    return celsius;
}

```

To emulate I2C_read() function with CppMock, we will use below moci_i2c.c file. As we learned before, we need to include “**CppUTestExt/MockSupport_c.h**” and use “mock_c()->” types.

```

#include <stdbool.h>
#include <stdint.h>
#include <stddef.h>
#include "CppUTestExt/MockSupport_c.h"

bool I2C_read(uint8_t dev_addr, uint8_t reg_addr, uint8_t *data, size_t length)
{
    // Record the actual call parameters and the output buffer
    return mock_c()->actualCall("I2C_read")
        ->withUnsignedIntParameters("dev_addr", dev_addr)
        ->withUnsignedIntParameters("reg_addr", reg_addr)
        ->withOutputParameter("data", data) // Points to the buffer to be filled
        ->withUnsignedIntParameters("length", (unsigned int)length)
        ->returnBoolValueOrDefault(false);
}

```

On each test of test_sensor.cpp, it assigns I2C value with “mock().expectOneCall”, and use mock->actualCall() through read_temperature_sensor() and I2C_read(). At the end, teardown() cleaner executes mock.checkExpectations(), and mock.clear().

```

#include "CppUTest/TestHarness.h"
#include "CppUTestExt/MockSupport.h"

extern "C" {
    #include "sensor.h"
}

TEST_GROUP(SendorTestGroup)
{
    void setup() override
    {
    }
    void teardown() override
    {
        mock().checkExpectations();
        mock().clear();
    }
};

```

test_sensor.cpp

```

TEST(SendorTestGroup, ReadTempSuccess)
{
    uint8_t mock_temp_raw = 25;

    mock().expectOneCall("I2C_read")
        .withParameter("dev_addr", 0x48)
        .withParameter("reg_addr", 0x01)
        .withParameter("length", 1)

```

```

        .withOutputParameterReturning("data",
            &mock_temp_raw, sizeof(mock_temp_raw))
        .andReturnValue(true);

    int temp = read_temperature_sensor();
    LONGS_EQUAL(25, temp);
}

TEST(SendorTestGroup, ReadTempFail)
{
    mock().expectOneCall("I2C_read")
        .ignoreOtherParameters()
        .andReturnValue(false);

    int temp = read_temperature_sensor();
    LONGS_EQUAL(-999, temp);
}

```

On ReadTempSuccess, we assign 25 value, and read back. On ReadTempFail, we intentionally, break I2C read routine, and then check the “-999” value.

On Makefile, we need below for CppMock library.

```

...
CPPUTEST_LDFLAGS += -L$(CPPUTEST_HOME)/lib
CPPUTEST_LDFLAGS += -lCppUTest -lCppUTestExt

# Run MakefileWorker.mk with the variables set in this Makefile
include $(CPPUTEST_HOME)/build/MakefileWorker.mk
...

```

If you need while Makefile and source code, clone “<https://github.com/vspauto/utest.git>”.